# Distributed Agentic Software Engineering Workbench

Self-Healing Software Factory - Local Framework & Groundbase Plan

Prepared for: Alfee | Scope: local-only code framework (no remote GitHub integration, no physical hardware)

Workspace: /Users/shahrin-a/private/Agentic SWE Workbench

## 1. Vision

Inspired by autonomous engineering frameworks such as Open SWE and MetaGPT, this project constructs a software development factory that ingests issue tickets, analyzes multi-file codebases, formulates modification blueprints, synthesizes code edits, and executes unit tests in isolated environments. When tests fail, a self-healing loop diagnoses failures and refines patches until verification constraints pass (or a hard iteration budget is exhausted).

## 2. Locked defaults (current build)

- Local only - orchestrator, sandbox, and fixtures run on the developer machine.
- No GitHub - tickets via CLI; results written under .runs/ (no remotes, PRs, or webhooks).
- No cloud sandbox required for groundbase - isolated worktrees under .runs/ + subprocess execution.
- MCP tool surface retained so a cloud sandbox provider can plug in later.
- Custom Open SWE / MetaGPT-inspired multi-agent graph (not a fork).
- Workbench language: Python 3.12+.

## 3. System architecture

Hierarchical multi-agent graph with decoupled execution environments.

### 3.1 Orchestration (LangGraph)

- Manager Agent - ingest ticket, normalize acceptance criteria, track phase/status.
- Planner Agent - parse codebase structure (AST graph), draft modification blueprint.
- Programmer Agent - synthesize exact file edits inside the sandbox and run tests.
- Diagnostic Agent - on failure, parse traces and produce a constrained repair brief.

Flow: Manager -> Planner -> Programmer -> (tests fail) Diagnostic loop -> Succeeded/Failed

### 3.2 Code execution layer

- LocalSandbox copies a source repo into .runs/{run_id}/workspace.
- Commands (install/test/shell) are cwd-bound to that worktree.
- Same tools are exposed via an MCP server (stdio) for agent tooling.
- MCP tools: sandbox_create, list/read/write files, run, run_tests, destroy.

### 3.3 Codebase inspection

- Python AST indexer builds a symbol + import dependency graph.
- Planner queries by keywords/symbols to avoid dumping entire repos into context.

### 3.4 Self-healing

- Capture pytest / compiler output from the sandbox.
- Diagnostic parses failing tests and assertion hints.
- DSPy-facing PatchRefiner stub (offline heuristic now; optional DSPy with LLM extras).

- Hard caps: max heal attempts and max files touched.

## 4. Repository layout

```
src/agentic_swe/
  cli.py                 Typer CLI (run / graph / mcp)
  config.py              Settings (.env + AGENTIC_SWE_*)
  models.py              Ticket, EditPlan, RunState, TestResult
  sandbox/               LocalSandbox, SandboxManager, MCP server
  code_intel/            AST indexer + DependencyGraph
  healing/               diagnostic + DSPy stubs
  orchestrator/          LangGraph factory graph + agent nodes
examples/fixture_repo/   Intentionally broken calculator for demos
tests/                   Unit/integration tests for the workbench
docs/                    Shareable plans (this PDF)
.runs/                   Per-run sandboxes + state.json artifacts
```

## 5. What you need (inventory)

### Physical

- Developer laptop only. No robots, sensors, or on-prem GPU required for the groundbase.

### Virtual / accounts (optional for offline groundbase)

- LLM API key (OpenAI/Anthropic/etc.) - optional; heuristic mode works without it.
- Optional later: E2B or other cloud sandbox if you leave local-only mode.
- Optional later: LangSmith for tracing.

### Software stack

- Python 3.12+ (Homebrew python@3.12 on this machine)
- langgraph, langchain-core, pydantic, typer, rich, mcp
- pytest / ruff for local verification
- Optional extras: langchain-openai, dspy-ai

### Local secrets (.env.example)

```
OPENAI_API_KEY=                # optional
OPENAI_MODEL=gpt-4o-mini
AGENTIC_SWE_RUNS_DIR=.runs
AGENTIC_SWE_MAX_HEAL_ATTEMPTS=5
AGENTIC_SWE_MAX_FILES_TOUCHED=8
AGENTIC_SWE_COMMAND_TIMEOUT_SEC=120
```

## 6. Delivery status (groundbase)

- Phase 0 - project skeleton, schemas, settings: DONE
- Phase 1 - local sandbox + CLI smoke on fixture_repo: DONE
- Phase 2 - LangGraph Manager/Planner/Programmer/Diagnostic: DONE
- Phase 3 - AST dependency graph + healing stubs with caps: DONE
- Out of scope for now: GitHub App, cloud sandboxes, web dashboard, TypeScript targets, hosting

## 7. How to run locally

```
cd "/Users/shahrin-a/private/Agentic SWE Workbench"
```

```
/opt/homebrew/bin/python3.12 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"

agentic-swe run \
  --repo examples/fixture_repo \
  --ticket "fix add so that 2+2 equals 4"

agentic-swe graph examples/fixture_repo
agentic-swe mcp   # stdio MCP sandbox server
```

Artifacts: .runs/{run_id}/workspace and .runs/{run_id}/state.json. The original fixture remains intentionally broken.

## 8. Risks & guardrails

- Unbounded heal loops - hard max iterations + spend/file caps.
- Context overflow - Planner must use graph queries, not full-repo dumps.
- Nondeterministic green - record exact test command + exit code in run state.
- Secret leakage - never inject host secrets into sandboxes unnecessarily.
- Scope creep - keep shipping thin local vertical slices before cloud/GitHub integrations.

## 9. Suggested next extensions (later)

- Optional LLM-backed Planner/Programmer when OPENAI_API_KEY is set.
- Human-in-the-loop plan approval interrupt before writes.
- Reviewer subagent before marking success.
- Pluggable cloud sandbox backend behind the same MCP tool names.
- Multi-language Tree-sitter indexing (TypeScript targets).